System Design Documentation

C Mini Project

Title Of Project:- ‘Conquer The world’

Group Information:

College: D. Y. Patil, Agricultural and Technical University, Talsande
.

Department: Department of Engineering and Technology.

Course: Bachelor in Computer Science and

Engineering.

Class: 2nd year. **Div**: A **Batch:A1**

Guidance: Mr. S.J. Salunkhe

Group: G-4 **Problem:** ICPC 2018 Problem-C

Introduction:

The purpose of this document is to communicate the design of a system or product to stakeholders. It is a living document that is updated throughout the development processto reflect changes in the design.

Target audience: Target Audience of this system document are project guide for the purpose of validation of this documentation.

Test plan includes self-made test cases by group members and will be carried out after completion of code of this project. Test plan will contain numerous test cases which will verify the reliability of this project.

Problem Statement:

In the virtual world of Conqueria, there are N territories represented as nodes on a graph. Each territory has a certain resource value and is connected to other territories by roads. We are tasked with developing a program that strategically conquers territories, maximizing the total resource value while minimizing the cost of conquering.

Consider a Conqueria map with 5 territories, each having a resource value. Teams start with a limited number of armies and must conquer territories strategically. The solution should output the sequence of conquered territories, the total resource value gained, and the efficient use of resources.

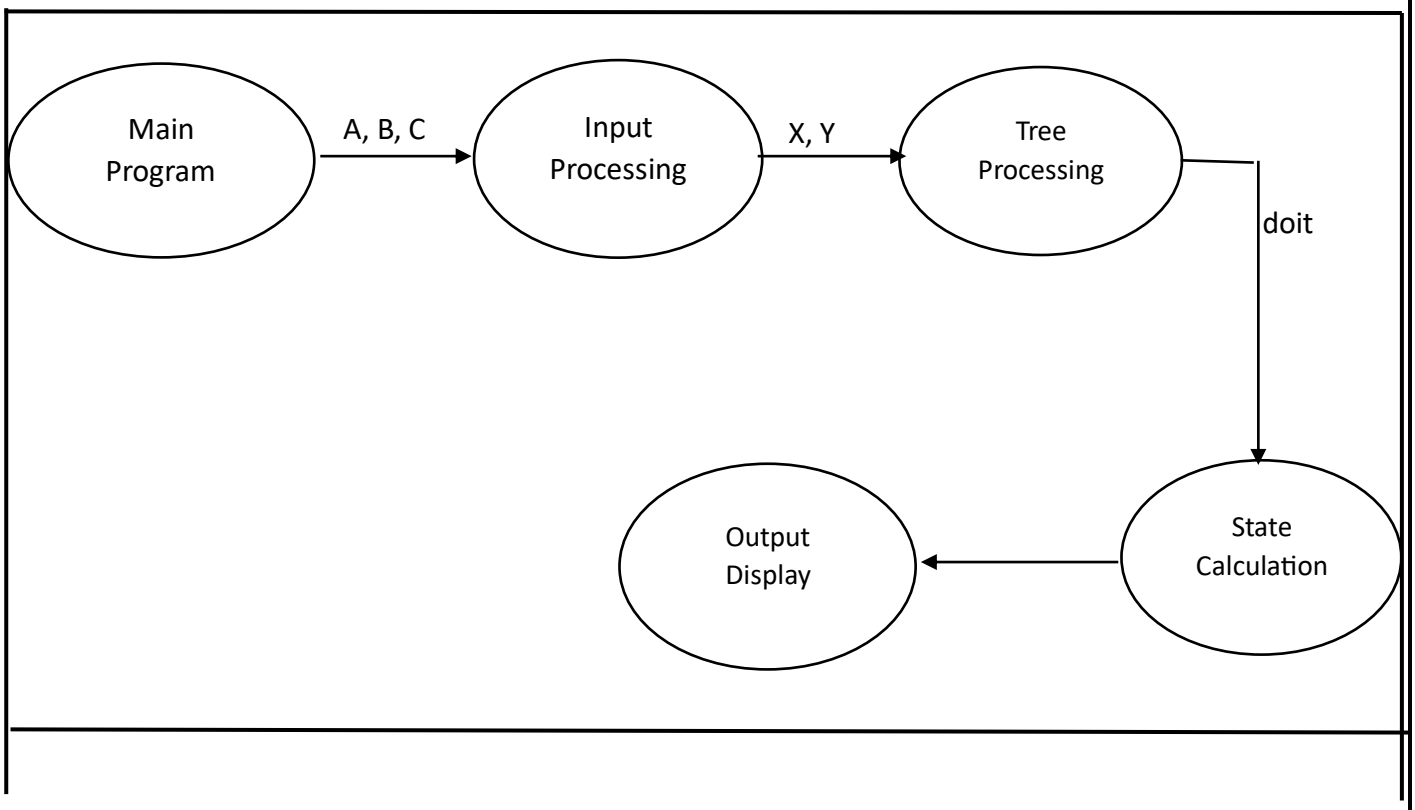
System Architecture:

C programming language is used to make this project.

The system architecture involves data structures for managing heaps and states, along with global arrays for representing the tree and node values.

The main function orchestrates the overall process of reading input, calculating the optimized state, and producing the final result.

Data Flow Diagram (Level 0):

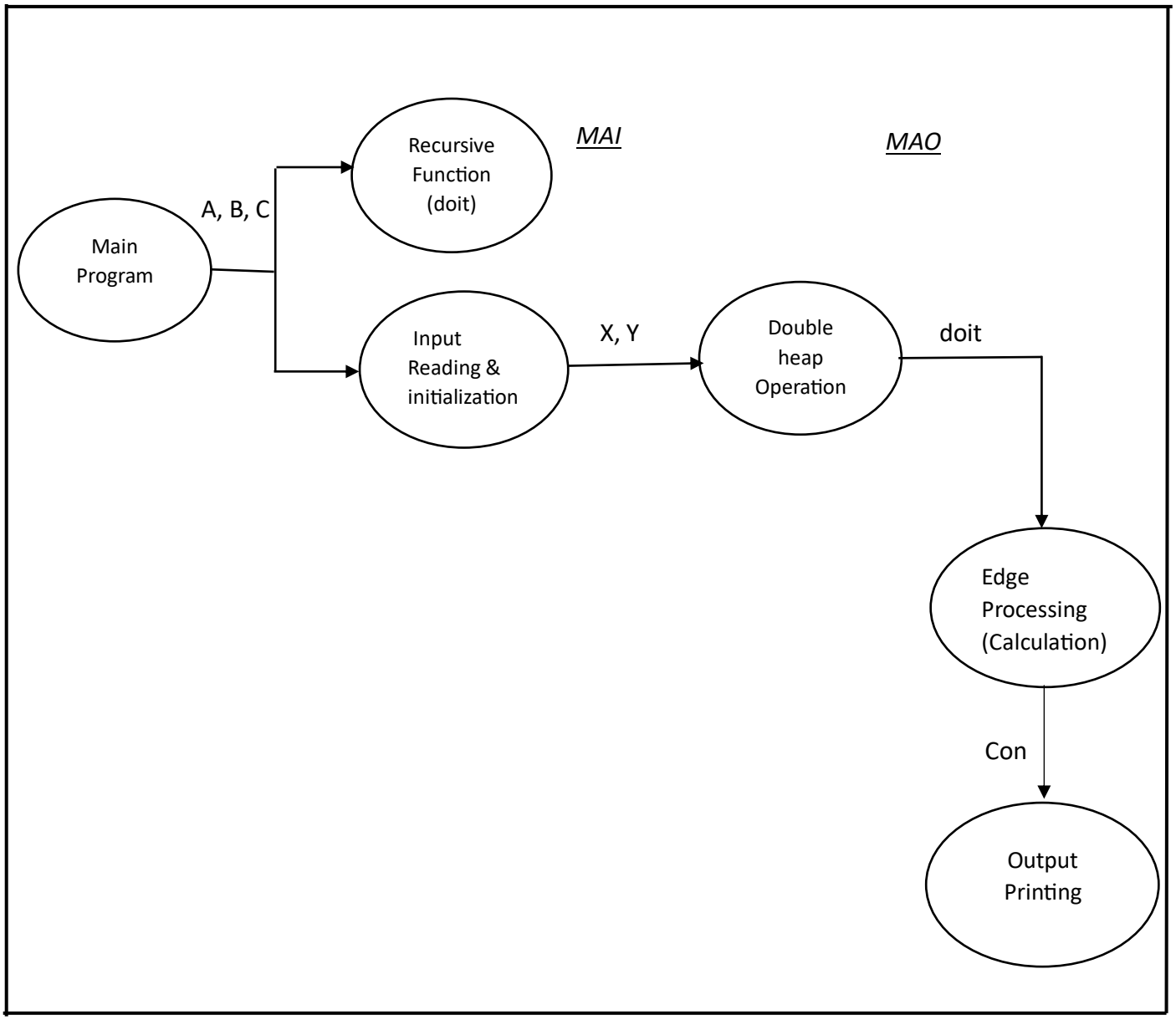


Explanation.

- **Main program:** The starting point of the program. It handles the overall flow of control.
- **Input processing:** Read the input data, including the number of nodes, tree edges, edge costs, and nods values.
- **Tree parsing:** Parses the tree structure, edge cost and node values, populates the necessary array, and prepare the data for the further processing.
- **State calculation:** Calls the **'idiot'** Function, which recursively calculate the state, including minimum increase **('minInc')**, **('base cost'),** and the double heap **('saving')** Structure.
- **Output display:** Print. The final result based on the calculated state. Considering, condition and adjusting the result for accuracy.

❖ **Identify the most Abstract input and output:-**

Data Flow Diagram (Level 1):



Components:

- The system follows a dynamic programming approach.
- The input is stored in an Array of structures.
- Different functions, modules (call by value + call by reference) are present to perform operations on the array and achieve the desired outcome.

Functions:

Initialize:

Initializes a **'DoubleHeap'** structure by setting base values and sizes.

shiftPartition :

Shift element between sets to maintain a specified partition size.

size:

Returns the total size of the double heat (sum of Size of set1and set2).

push:

Pushes an element into the double heap based on certain condition.

pop:

Pops removes an element from the double heap.

top:

Retrives the top element from the double heap.

pruneNeg:

Remove negative element from the double heap.

doit:

Recursive function that calculates the state of a node in the tree.

Traverses the tree and calculates the minimum increase, base cost, and manages saving using a **'DoubleHeap'.**

main:

Read inputs for the number of node **('N')** tree ages, age cost, and node value (X and Y) from standard input.

Calls the **'doit'** function to calculate the state and prints the final result.

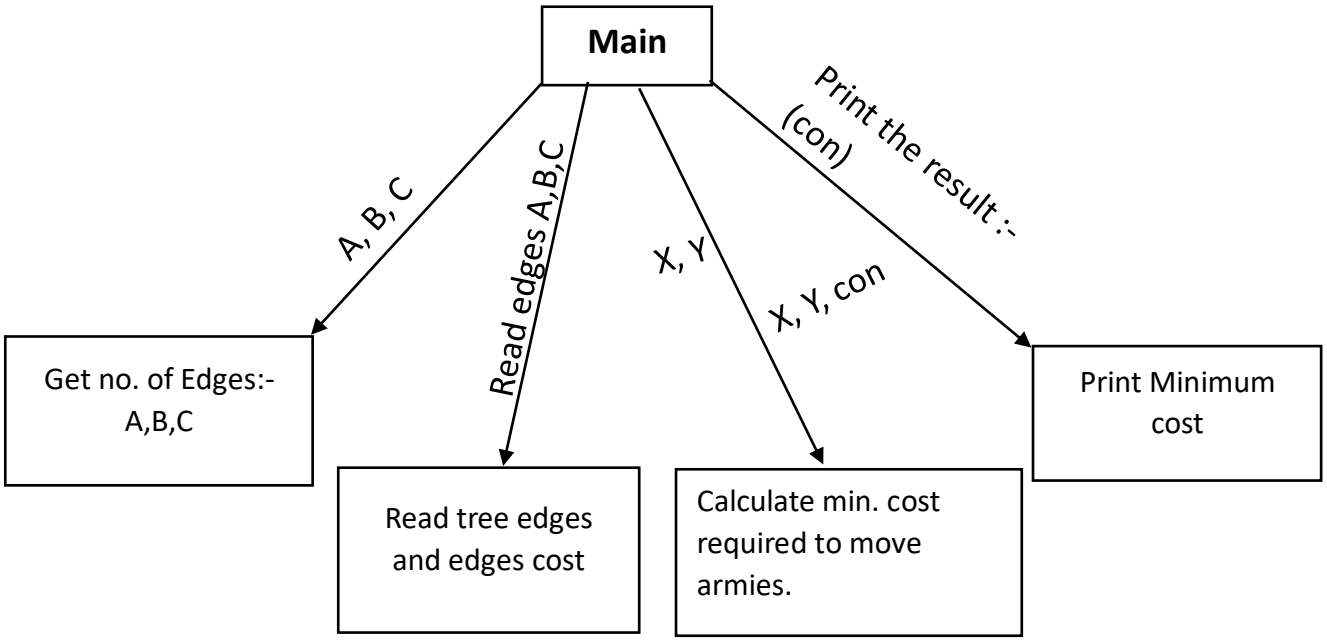


Fig c. First Level Factoring

Process:

Goal:

The main goal of the program is to calculate the minimum cost of modifying a given tree structure, where each node has associated value X and Y.

The program optimizes the modification to minimize the increase in value Y while considering associated cost and specific condition. The result represents the minimum cost required to achieve the desired modification.

Requirements:

a) **Non-Functional Requirements:**

1. Environment: Windows OS.

2. MINGW GCC compiler must be installed in order to run the .exe file provided to you as the program.

3. Application must be able to handle up-to 100k inputs. With each input containing digits ranging from 0 to 10^9.

b) Functional requirements :-

1 User must give input to program in order to perform its exhibition.

2 We should take care of invalid Input.

3 Based on valid inputs, User must get correct output (i.e minimum cost required

Sr. No.	Name	Roll No.	Sign
1	**Aary Satardekar**	01	
2	**Abdulwahid Jamadar**	03	
3	**Ajim Nadaf**	14	
4	**Akshay Sanade**	18	
5	**Ankit purohit**	24	

to move armies from edges.

Group Members :-

Mr. S. J. Salunkhe
(Project Guide)

Mr. S. A. Kumbhar
(Mini Project Co-ordinator)